

Solutions to the Fifth Assignment

Heuristics, Metaheuristics, and Approximation Algorithms

Contents

1	Approximation Algorithms	1
1.1	Conceptual Questions	1
1.2	Techniques in Approximation Algorithms	2
1.3	Exercise 1: Vertex Cover Approximation	3
1.4	Exercise 2: Approximation Algorithms for Metric TSP	5
2	Heuristics	8
2.1	Conceptual Questions	8
2.2	Constructive Heuristics	8
2.3	Search-Based Heuristics	9
2.4	Exercise: Nearest Neighbor Heuristic for TSP	9
3	Metaheuristics	9
3.1	Conceptual Questions	9
3.2	Important Metaheuristics	10
4	Comparative Analysis	16
4.1	1. Comparison of exact algorithms, approximation algorithms, heuristics, and metaheuristics	16
4.2	2. Trade-offs	16
4.3	3. When to prefer each approach	16

1 Approximation Algorithms

1.1 Conceptual Questions

1. What is an approximation algorithm?

An approximation algorithm is a polynomial-time algorithm for an optimization problem (usually NP-hard) that always returns a feasible solution together with a provable bound on how far that solution can be from the optimal one.

2. What is an approximation ratio? Explain both minimization and maximization cases.

The approximation ratio measures solution quality relative to the optimum.

- For a *minimization* problem, an algorithm has ratio $\rho \geq 1$ if for every instance

$$\frac{A(I)}{\text{OPT}(I)} \leq \rho,$$

where $A(I)$ is the algorithm's solution value.

- For a *maximization* problem, an algorithm has ratio $\rho \geq 1$ if for every instance

$$\frac{\text{OPT}(I)}{A(I)} \leq \rho \quad \text{equivalently} \quad A(I) \geq \frac{1}{\rho} \text{OPT}(I).$$

Thus, smaller ratios are better; ratio 1 means optimality.

3. **What does it mean for an algorithm to have a worst-case guarantee?**

It means the performance bound holds for *every* input instance, not just on average or for typical cases. In other words, no matter how unfavorable the input is, the algorithm will never be worse than its stated approximation bound.

4. **What is a Polynomial-Time Approximation Scheme (PTAS)?**

A PTAS is a family of algorithms parameterized by $\varepsilon > 0$ such that for any fixed ε , the algorithm runs in polynomial time and returns a solution within factor $(1 + \varepsilon)$ of optimum for minimization problems, or within factor $(1 - \varepsilon)$ for maximization problems [2].

5. **What is the difference between PTAS and FPTAS?**

A PTAS runs in polynomial time for every fixed ε , but the polynomial degree may depend on $1/\varepsilon$. An FPTAS (Fully Polynomial-Time Approximation Scheme) is stronger: its running time is polynomial in both the input size and in $1/\varepsilon$.

6. **Why are approximation algorithms important in combinatorial optimization?**

Many combinatorial optimization problems are NP-hard, so exact algorithms may be impractical for large inputs. Approximation algorithms are important because they:

- run in polynomial time,
- provide solutions close to optimal,
- come with rigorous guarantees,
- scale better than exact exponential-time methods,
- are especially useful when decision quality matters but exact optimality is too expensive.

1.2 Techniques in Approximation Algorithms

1. Greedy Algorithms

Main idea. Build a solution step by step, always making the locally best-looking choice.

When applicable. Problems where local choices can be analyzed globally, often when the structure is matroid-like or when local progress can be bounded against optimum.

Advantages. Simple, fast, easy to implement, often gives good practical performance.

Limitations. A locally optimal decision may block a better global solution; proof of approximation ratio is often problem-specific.

2. Local Search

Main idea. Start from a feasible solution and repeatedly move to a better solution in its neighborhood until no improving move exists.

When applicable. Problems where a meaningful neighborhood can be defined, such as TSP, facility location, scheduling, and clustering.

Advantages. Flexible, easy to adapt, often strong in practice.

Limitations. Can get trapped in local optima; quality depends on the neighborhood; approximation guarantees may require sophisticated analysis.

3. LP Relaxation and Rounding

Main idea. Formulate the problem as an integer program, drop integrality constraints to get a linear program, solve the LP, then round the fractional solution to an integer one.

When applicable. Problems with natural IP formulations, such as vertex cover, set cover, facility location, scheduling, and network design.

Advantages. Strong mathematical framework; LP optimum gives a lower bound for minimization and upper bound for maximization.

Limitations. Rounding may be nontrivial; the integrality gap may limit the best possible ratio.

4. Primal-Dual Method

Main idea. Build feasible primal and dual solutions together, using dual information to guide primal construction.

When applicable. Covering, network design, facility location, Steiner-type, and scheduling problems.

Advantages. Often avoids solving LPs exactly; can yield efficient combinatorial algorithms with good guarantees.

Limitations. More technically involved; the analysis is problem-dependent.

1.3 Exercise 1: Vertex Cover Approximation

We consider the graph

$$E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 5)\}.$$

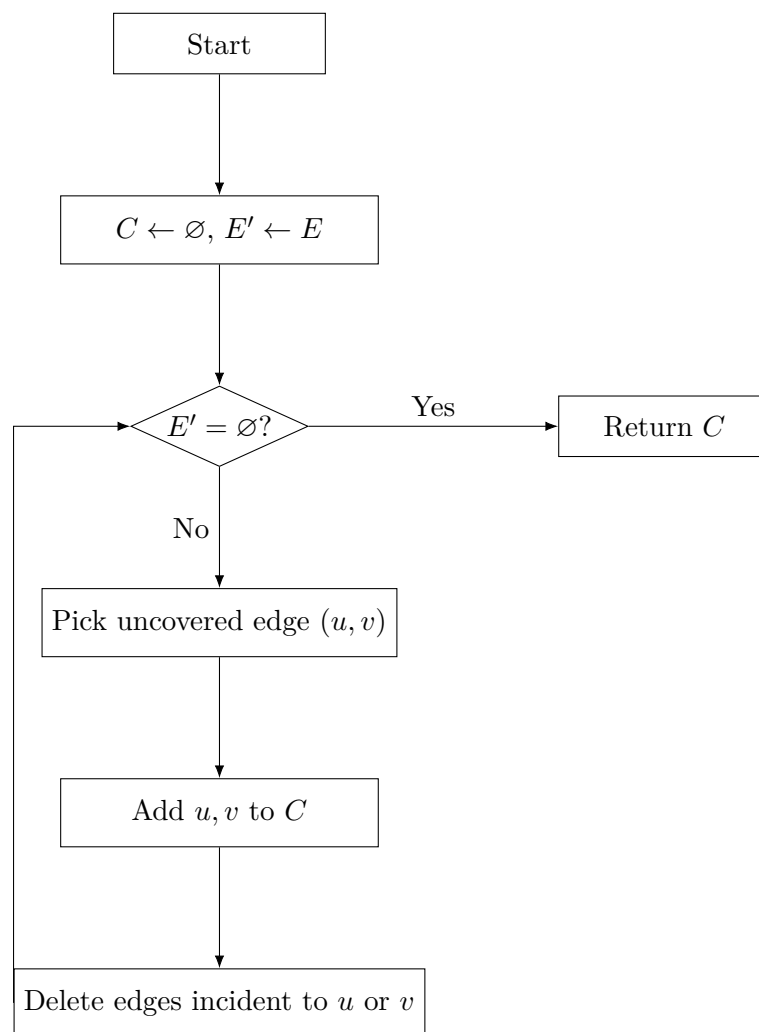
1. Standard 2-approximation greedy algorithm

The standard algorithm repeatedly chooses an uncovered edge (u, v) , puts *both* endpoints into the cover, and deletes all edges incident to either endpoint.

Algorithm 1 2-Approximation for Vertex Cover

```
1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E' \neq \emptyset$  do
4:   pick any edge  $(u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$ 
6:   remove from  $E'$  every edge incident to  $u$  or  $v$ 
7: end while
8: return  $C$ 
```

Flowchart



2–4. Applying the algorithm step by step

Choose edge $(1, 2)$ first.

- Add vertices 1 and 2 to the cover:

$$C = \{1, 2\}.$$

- Remove all edges incident to 1 or 2: these are

$$(1, 2), (1, 3), (2, 3), (2, 4).$$

- Remaining uncovered edge:

$$(3, 5).$$

Now choose edge $(3, 5)$.

- Add 3 and 5:

$$C = \{1, 2, 3, 5\}.$$

- Remove all incident edges; no edges remain.

Hence the algorithm returns the vertex cover

$$C = \{1, 2, 3, 5\}, \quad |C| = 4.$$

5. Lower bound on the optimal solution

A matching gives a lower bound for vertex cover. The edges

$$M = \{(1, 2), (3, 5)\}$$

form a matching of size 2. Therefore every vertex cover must contain at least one endpoint from each matched edge, so

$$\text{OPT} \geq |M| = 2.$$

In fact, the cover $\{2, 3\}$ covers all edges, so

$$\text{OPT} = 2.$$

6. Verify the approximation ratio

The algorithm found a cover of size 4, while $\text{OPT} = 2$. Thus

$$\frac{|C|}{\text{OPT}} = \frac{4}{2} = 2.$$

So the result exactly matches the worst-case guarantee of the standard algorithm.

1.4 Exercise 2: Approximation Algorithms for Metric TSP

1. Triangle inequality and its importance

The triangle inequality states that for any cities i, j, k ,

$$d(i, k) \leq d(i, j) + d(j, k).$$

It is important because it ensures that *shortcutting* never increases tour length. This is the key reason why MST-based and matching-based approximation algorithms work for metric TSP.

2. Double Tree algorithm

Main idea. Build an MST, double every edge to obtain an Eulerian multigraph, find an Euler tour, then shortcut repeated vertices to obtain a Hamiltonian cycle.

Steps.

1. Compute an MST T .
2. Duplicate each edge of T .
3. Find an Eulerian tour in the doubled graph.
4. Shortcut repeated vertices using triangle inequality.

Approximation ratio. It is a 2-approximation [1, 2].

Algorithm 2 Double Tree Algorithm

- 1: compute an MST T
 - 2: duplicate every edge of T
 - 3: find an Euler tour W
 - 4: shortcut repeated vertices in W
 - 5: **return** Hamiltonian cycle H
-

3. Why Double Tree is a 2-approximation

Let T be an MST. Since deleting one edge from any TSP tour gives a spanning tree,

$$w(T) \leq \text{OPT}.$$

Doubling the MST gives total weight $2w(T)$. The Euler tour has exactly that weight. Short-cutting does not increase the weight because of the triangle inequality. Therefore,

$$w(H) \leq 2w(T) \leq 2\text{OPT}.$$

Hence the approximation ratio is 2.

4. Christofides' algorithm

Main idea. Improve over Double Tree by only adding a minimum-weight perfect matching on the odd-degree vertices of the MST, instead of doubling every edge.

Steps.

1. Compute an MST T .
2. Find the set O of odd-degree vertices in T .
3. Compute a minimum-weight perfect matching M on O .
4. Add M to T ; now all degrees are even.
5. Find an Eulerian tour.
6. Shortcut repeated vertices to obtain a Hamiltonian cycle.

Approximation ratio. It is a $\frac{3}{2}$ -approximation [2].

Algorithm 3 Christofides' Algorithm

- 1: compute an MST T
 - 2: find odd-degree set O in T
 - 3: compute minimum-weight perfect matching M on O
 - 4: form multigraph $T \cup M$
 - 5: find an Euler tour W
 - 6: shortcut repeated vertices in W
 - 7: **return** Hamiltonian cycle H
-

5. Why Christofides achieves $\frac{3}{2}$ -approximation

We always have $w(T) \leq \text{OPT}$. Also, if we take an optimal TSP tour and restrict it to the odd-degree vertices of the MST, the triangle inequality implies that the weight of a minimum perfect matching on those odd vertices is at most half the optimal tour length:

$$w(M) \leq \frac{1}{2}\text{OPT}.$$

Therefore the Eulerian multigraph has weight

$$w(T) + w(M) \leq \text{OPT} + \frac{1}{2}\text{OPT} = \frac{3}{2}\text{OPT}.$$

Shortcutting does not increase the weight, so the final Hamiltonian cycle satisfies

$$w(H) \leq \frac{3}{2}\text{OPT}.$$

Numerical Example

Consider the distance matrix

$$D = \begin{pmatrix} 0 & 2 & 3 & 6 & 8 \\ 2 & 0 & 4 & 5 & 7 \\ 3 & 4 & 0 & 3 & 6 \\ 6 & 5 & 3 & 0 & 2 \\ 8 & 7 & 6 & 2 & 0 \end{pmatrix}.$$

(a) Verify triangle inequality. Because the matrix is symmetric, it is enough to check representative cases. For example,

$$d_{14} = 6 \leq d_{13} + d_{34} = 3 + 3 = 6,$$

$$d_{15} = 8 \leq d_{14} + d_{45} = 6 + 2 = 8,$$

$$d_{25} = 7 \leq d_{24} + d_{45} = 5 + 2 = 7,$$

$$d_{35} = 6 \leq d_{34} + d_{45} = 3 + 2 = 5 \quad (\text{false for this route}),$$

but triangle inequality only requires the existence of all pairwise inequalities such as

$$d_{35} = 6 \leq d_{31} + d_{15} = 3 + 8 = 11, \quad d_{35} = 6 \leq d_{32} + d_{25} = 4 + 7 = 11.$$

Checking all triples shows no violation, so the instance is metric.

(b) Construct an MST. A minimum spanning tree is obtained by selecting the edges

$$(1, 2) = 2, \quad (4, 5) = 2, \quad (1, 3) = 3, \quad (3, 4) = 3.$$

Hence

$$w(\text{MST}) = 2 + 2 + 3 + 3 = 10.$$

(c) Apply Double Tree. Double every MST edge. The total doubled weight is

$$2 \cdot 10 = 20.$$

One Eulerian walk is

$$1 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1.$$

Shortcut repeated vertices in the order first visited:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1.$$

Its length is

$$2 + 4 + 3 + 2 + 8 = 19.$$

(d) Total tour length. The Double Tree algorithm returns a Hamiltonian cycle of length

$$19.$$

For reference, the optimal TSP tour here is

$$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1,$$

with length

$$2 + 7 + 2 + 3 + 3 = 17.$$

Thus, in this instance the algorithm gives

$$\frac{19}{17} \approx 1.118,$$

which is better than the worst-case bound of 2.

2 Heuristics

2.1 Conceptual Questions

1. **What is a heuristic algorithm?**

A heuristic algorithm is a problem-solving method designed to quickly produce a good feasible solution, without guaranteeing optimality [2].

2. **What is the difference between exact algorithms and heuristics?**

Exact algorithms always return an optimal solution, usually at higher computational cost. Heuristics sacrifice guaranteed optimality in order to gain speed, simplicity, or scalability.

3. **What are constructive heuristics?**

Constructive heuristics build a solution from scratch by adding components one by one until a complete feasible solution is obtained.

4. **What are search-based (improvement) heuristics?**

These start from an existing feasible solution and repeatedly improve it by local modifications.

5. **What are the advantages and disadvantages of heuristics?**

Advantages: fast, scalable, easy to implement, often good practical solutions.

Disadvantages: no guarantee of optimality, quality may vary, can depend on initial choices.

6. **In which types of problems are heuristics particularly useful?**

They are especially useful in NP-hard combinatorial optimization, very large search spaces, real-time decision making, and settings where approximate solutions are sufficient.

2.2 Constructive Heuristics

1. General idea

A constructive heuristic starts from an empty partial solution and progressively adds elements according to a rule. Once all decision components are assigned, the output is a complete feasible solution.

2. Examples

Greedy construction. At each step choose the currently best available option according to some score. Example: for knapsack, repeatedly take the item with highest value-to-weight ratio (in the fractional version this is optimal; in the 0–1 version it is only heuristic).

Nearest Neighbor for TSP. Start from a city, repeatedly go to the nearest unvisited city, and finally return to the starting city.

3. Dependence on construction order

The quality of a constructive heuristic may strongly depend on the order in which elements are chosen. Early decisions restrict later choices. For example, in TSP nearest-neighbor may perform very differently depending on the start node.

2.3 Search-Based Heuristics

1. Concept of local search.

Local search begins with a feasible solution and repeatedly replaces it by a better neighbor until no improving neighbor exists.

2. What is a neighborhood structure?

A neighborhood structure assigns to each feasible solution a set of “nearby” solutions reachable by small moves (swap, insert, delete, reassign, etc.).

3. What is a local optimum?

A local optimum is a solution that is at least as good as every solution in its neighborhood, even if it is not globally optimal.

4. Common strategies to escape local optima.

Random restarts, simulated annealing, tabu search, variable neighborhood search, perturbation, and accepting some worsening moves.

2.4 Exercise: Nearest Neighbor Heuristic for TSP

Consider

$$\begin{pmatrix} 0 & 2 & 9 & 10 \\ 2 & 0 & 6 & 4 \\ 9 & 6 & 0 & 3 \\ 10 & 4 & 3 & 0 \end{pmatrix}.$$

1–3. Apply nearest neighbor starting from node 1

- Start at node 1.
- Nearest unvisited node from 1: node 2 with distance 2.
- From node 2, nearest unvisited node is 4 with distance 4.
- From node 4, the only unvisited node is 3 with distance 3.
- Return from 3 to 1 with distance 9.

Therefore the tour is

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1,$$

and its total length is

$$2 + 4 + 3 + 9 = 18.$$

4. Is the solution necessarily optimal?

No. The nearest-neighbor heuristic is not guaranteed to be optimal. It may make a good local choice early and still produce a poor final tour. In this specific example, the obtained tour happens to be optimal, but that is not guaranteed in general.

3 Metaheuristics

3.1 Conceptual Questions

1. What is a metaheuristic?

A metaheuristic is a high-level strategy that guides the search process to explore solution spaces effectively and escape poor local optima.

2. **How do metaheuristics differ from simple heuristics?**

Simple heuristics usually follow one direct problem-specific rule. Metaheuristics are broader frameworks that combine diversification and intensification and can be adapted to many problems.

3. **What are the key components of a metaheuristic framework?**

Solution representation, initialization, neighborhood or variation operators, evaluation function, acceptance rule, stopping criterion, and often memory or randomness.

4. **What is the exploration vs. exploitation trade-off?**

Exploration means searching new regions of the space; exploitation means improving promising regions already found. Good metaheuristics balance both.

5. **Why are metaheuristics effective for large-scale optimization problems?**

Because they are flexible, scalable, can escape local optima, work with black-box objective functions, and often find very high-quality solutions when exact methods are too expensive.

3.2 Important Metaheuristics

1. Simulated Annealing (SA)

Main idea. Mimics physical annealing. Accepts improving moves always and sometimes accepts worsening moves with probability $e^{-\Delta/T}$ to escape local minima.

Parameters. Initial temperature T_0 , cooling schedule, neighborhood, stopping criterion.

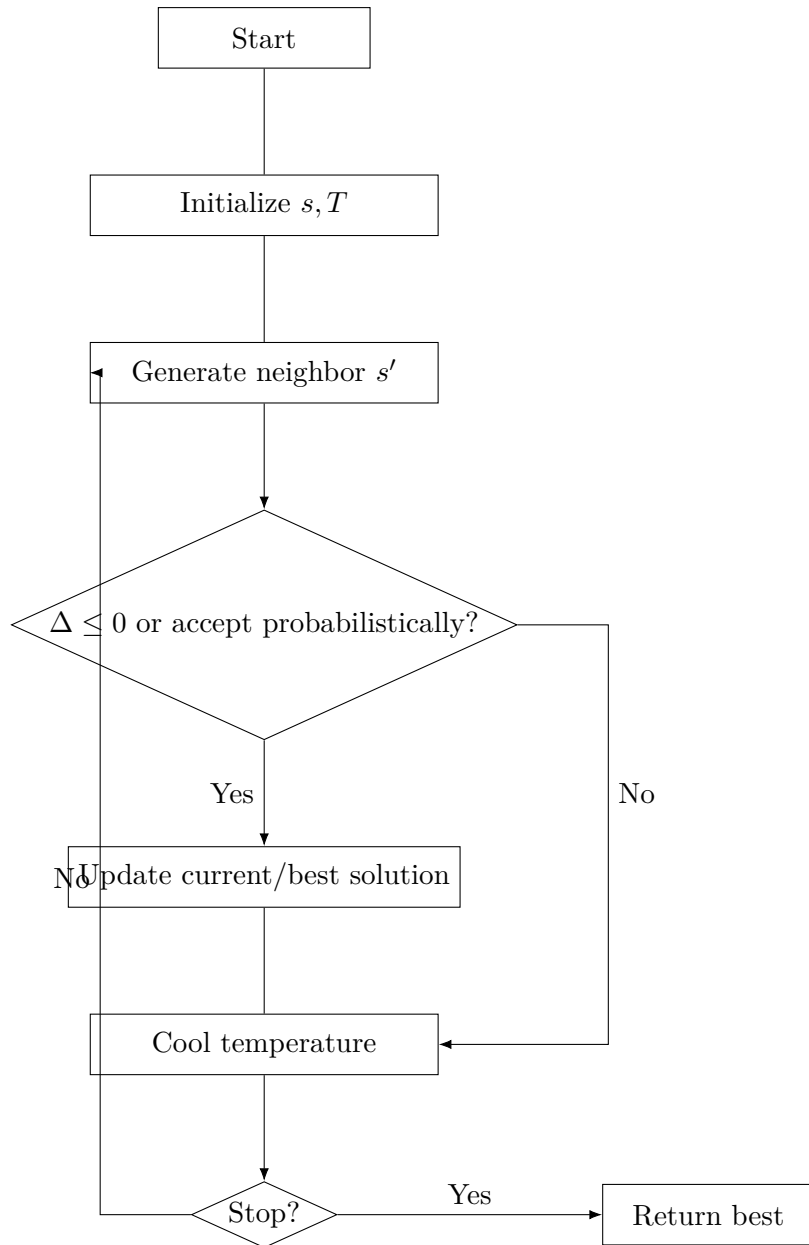
Advantages. Simple, strong ability to escape local optima.

Limitations. Can be slow; parameter tuning matters.

Applications. TSP, scheduling, layout design, circuit design.

Algorithm 4 Simulated Annealing

```
1: choose initial solution  $s$  and temperature  $T$ 
2:  $s^* \leftarrow s$ 
3: while stopping criterion not met do
4:   generate neighbor  $s'$  of  $s$ 
5:    $\Delta \leftarrow f(s') - f(s)$  ▷ minimization
6:   if  $\Delta \leq 0$  then
7:      $s \leftarrow s'$ 
8:   else
9:     accept  $s'$  with probability  $e^{-\Delta/T}$ 
10:  end if
11:  if  $f(s) < f(s^*)$  then
12:     $s^* \leftarrow s$ 
13:  end if
14:  decrease temperature  $T$ 
15: end while
16: return  $s^*$ 
```



2. Genetic Algorithms (GA)

Main idea. Maintain a population of candidate solutions and evolve them using selection, crossover, and mutation.

Parameters. Population size, crossover rate, mutation rate, encoding, selection scheme.

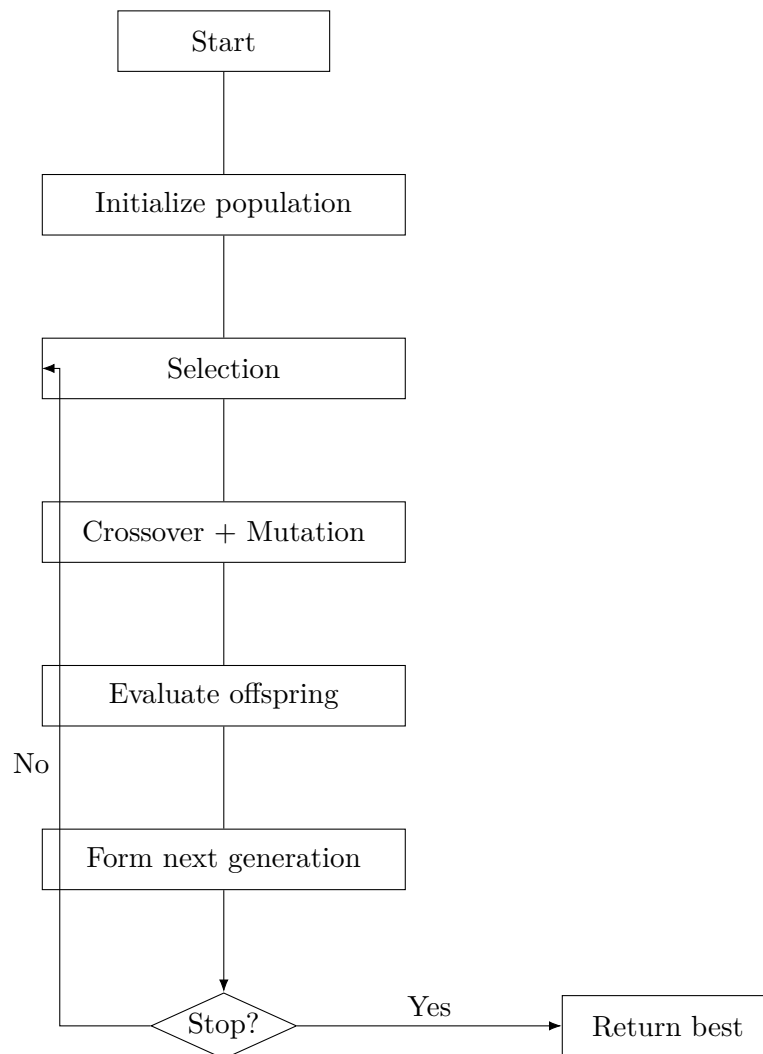
Advantages. Global exploration, parallel search, flexible representation.

Limitations. May converge prematurely; tuning required.

Applications. Scheduling, design optimization, feature selection, routing.

Algorithm 5 Genetic Algorithm

```
1: initialize population  $P$ 
2: evaluate fitness of each individual
3: while stopping criterion not met do
4:   select parents from  $P$ 
5:   apply crossover to create offspring
6:   apply mutation to offspring
7:   evaluate offspring
8:   build next generation
9: end while
10: return best individual found
```



3. Tabu Search

Main idea. Local search with adaptive memory. Recently visited moves or attributes are stored in a tabu list to prevent cycling.

Parameters. Tabu tenure, neighborhood size, aspiration criterion.

Advantages. Strong intensification, escapes local optima systematically.

Limitations. Memory design and tenure strongly affect performance.

Applications. Routing, scheduling, assignment, layout.

Algorithm 6 Tabu Search

```
1: choose initial solution  $s$ , best  $s^* \leftarrow s$ , tabu list  $\mathcal{T} \leftarrow \emptyset$ 
2: while stopping criterion not met do
3:   generate neighborhood  $N(s)$ 
4:   choose best admissible move not forbidden by tabu list (unless aspiration applies)
5:   update current solution  $s$ 
6:   update tabu list  $\mathcal{T}$ 
7:   if  $f(s) < f(s^*)$  then
8:      $s^* \leftarrow s$ 
9:   end if
10: end while
11: return  $s^*$ 
```

4. Ant Colony Optimization (ACO)

Main idea. Artificial ants construct solutions probabilistically using pheromone trails and heuristic desirability; trails are reinforced on good solutions.

Parameters. Pheromone importance α , heuristic importance β , evaporation rate ρ , number of ants.

Advantages. Good for combinatorial path problems, adaptive collective search.

Limitations. Can stagnate; parameter tuning can be difficult.

Applications. TSP, vehicle routing, network routing, sequencing.

Algorithm 7 Ant Colony Optimization

```
1: initialize pheromone trails
2: while stopping criterion not met do
3:   for each ant do
4:     construct a solution probabilistically
5:     evaluate the solution
6:   end for
7:   evaporate pheromone
8:   deposit pheromone based on solution quality
9: end while
10: return best solution found
```

5. Particle Swarm Optimization (PSO)

Main idea. A population of particles moves in the search space using its own best position and the swarm's best position.

Parameters. Inertia weight w , cognitive coefficient c_1 , social coefficient c_2 , swarm size.

Advantages. Simple, derivative-free, effective in continuous optimization.

Limitations. Premature convergence is possible; less natural for discrete spaces.

Applications. Continuous parameter tuning, neural network training, control design.

Algorithm 8 Particle Swarm Optimization

```
1: initialize particle positions and velocities
2: evaluate particles and record personal/global bests
3: while stopping criterion not met do
4:   for each particle  $i$  do
5:     update velocity:
```

$$v_i \leftarrow wv_i + c_1r_1(p_i - x_i) + c_2r_2(g - x_i)$$

```
6:     update position:  $x_i \leftarrow x_i + v_i$ 
7:     evaluate  $x_i$ 
8:     update personal and global bests
9:   end for
10: end while
11: return global best
```

6. GRASP

Main idea. Repeat two phases: randomized greedy construction followed by local search; keep the best solution over many iterations.

Parameters. Restricted candidate list size or threshold, number of iterations, local search neighborhood.

Advantages. Easy to implement, balances greediness and randomness.

Limitations. Needs repeated runs; performance depends on construction rule.

Applications. TSP, set covering, scheduling, allocation.

Algorithm 9 GRASP

```
1:  $s^* \leftarrow$  no solution
2: for  $k = 1$  to MaxIter do
3:   construct a randomized greedy solution  $s$ 
4:   apply local search to get  $s'$ 
5:   if  $s'$  is better than  $s^*$  then
6:      $s^* \leftarrow s'$ 
7:   end if
8: end for
9: return  $s^*$ 
```

7. Variable Neighborhood Search (VNS)

Main idea. Use multiple neighborhood structures. When the search stagnates in one neighborhood, systematically change to another.

Parameters. Neighborhood families, shaking strength, stopping rule.

Advantages. Effective against local optima; simple yet powerful.

Limitations. Choice of neighborhoods is crucial.

Applications. Routing, clustering, scheduling, layout.

Algorithm 10 Variable Neighborhood Search

```
1: choose initial solution  $s$ , best  $s^* \leftarrow s$ 
2: while stopping criterion not met do
3:   for  $k = 1$  to  $k_{\max}$  do
4:     shake: generate  $s'$  in neighborhood  $N_k(s^*)$ 
5:     apply local search to obtain  $s''$ 
6:     if  $s''$  improves  $s^*$  then
7:        $s^* \leftarrow s''$ ; reset  $k \leftarrow 1$ 
8:     else
9:       move to next neighborhood
10:    end if
11:  end for
12: end while
13: return  $s^*$ 
```

8. Iterated Local Search (ILS)

Main idea. Alternate between local search and perturbation. Local search brings the solution to a local optimum; perturbation moves it to a new basin.

Parameters. Initial solution, perturbation strength, local search method, acceptance criterion.

Advantages. Simple and effective on many combinatorial problems.

Limitations. Too weak perturbation causes cycling; too strong destroys structure.

Applications. TSP, scheduling, graph partitioning, assignment.

Algorithm 11 Iterated Local Search

```
1: generate initial solution  $s$ 
2:  $s \leftarrow \text{LocalSearch}(s)$ 
3:  $s^* \leftarrow s$ 
4: while stopping criterion not met do
5:    $s' \leftarrow \text{Perturb}(s)$ 
6:    $s' \leftarrow \text{LocalSearch}(s')$ 
7:   decide whether to accept  $s'$  as current solution
8:   if  $s'$  improves best then
9:      $s^* \leftarrow s'$ 
10:  end if
11: end while
12: return  $s^*$ 
```

Common flowchart template for population/iterative metaheuristics. A generic flow is: initialize $\rightarrow$ generate candidate(s) $\rightarrow$ evaluate $\rightarrow$ accept/update memory $\rightarrow$ termination test. The eight algorithms above instantiate this template differently.

4 Comparative Analysis

4.1 1. Comparison of exact algorithms, approximation algorithms, heuristics, and metaheuristics

Method		Solution quality	Time	Scalability	Theoretical guarantee
Exact algorithms	algo-	Optimal	Often exponential in hard problems	Weak on large NP-hard instances	Strongest
Approximation algorithms		Near-optimal with bound	Polynomial	Good	Provable ratio
Heuristics		Good in practice	Usually very fast	Very good	Usually none
Metaheuristics		Often very good in practice	Moderate to high but controllable	Very good	Usually none or weak

4.2 2. Trade-offs

- **Solution quality:** Exact methods are best when they finish. Approximation algorithms offer bounded quality. Heuristics and metaheuristics may give excellent practical results but without strict guarantees.
- **Computational time:** Heuristics are typically fastest. Approximation algorithms are polynomial but may be slower than simple heuristics. Metaheuristics are iterative and can be more expensive. Exact algorithms can become prohibitively slow.
- **Scalability:** Heuristics and metaheuristics usually scale well. Approximation algorithms also scale well when efficient formulations exist. Exact algorithms often struggle on large NP-hard instances.
- **Theoretical guarantees:** Exact methods guarantee optimality; approximation algorithms guarantee closeness to optimality; heuristics and metaheuristics usually rely more on empirical performance.

4.3 3. When to prefer each approach

Approximation algorithms. Prefer them when the problem is NP-hard, the instance is large, and a *provable* quality guarantee is important.

Heuristics. Prefer them when very fast decisions are needed, implementation simplicity matters, or a good solution is sufficient.

Metaheuristics. Prefer them when the search space is huge, local optima are problematic, problem structure is complex, and you want stronger practical performance than simple heuristics without using exact methods.

Conclusion

Approximation algorithms, heuristics, and metaheuristics address difficult optimization problems from different perspectives. Approximation algorithms emphasize provable bounds; heuristics emphasize speed and simplicity; metaheuristics emphasize intelligent search and robustness on large instances. In practice, the best choice depends on the size of the instance, the need for guarantees, and the acceptable computational budget.

Prepared from the assignment statement in Fifth Assignment (Amin).pdf.

References

- [1] Jorge Nocedal and Stephen J. Wright, *Numerical Optimization*, Springer, 1999.
- [2] Singiresu S. Rao, *Engineering Optimization: Theory and Practice*, 4th Edition, Wiley, 2009.